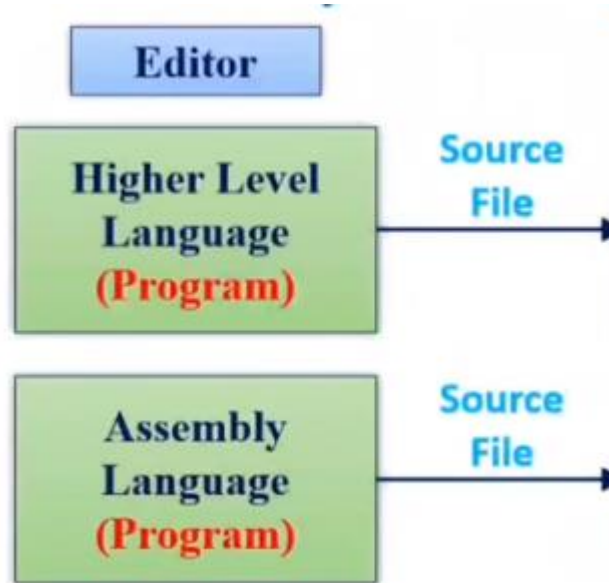


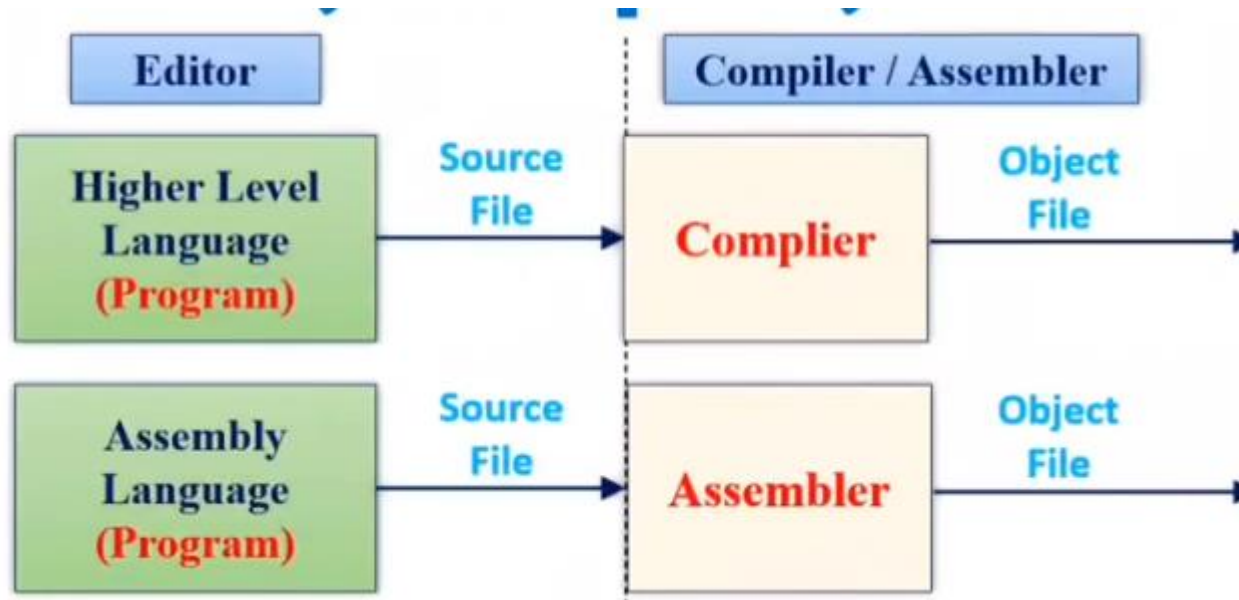
Lecture 4

System development tool



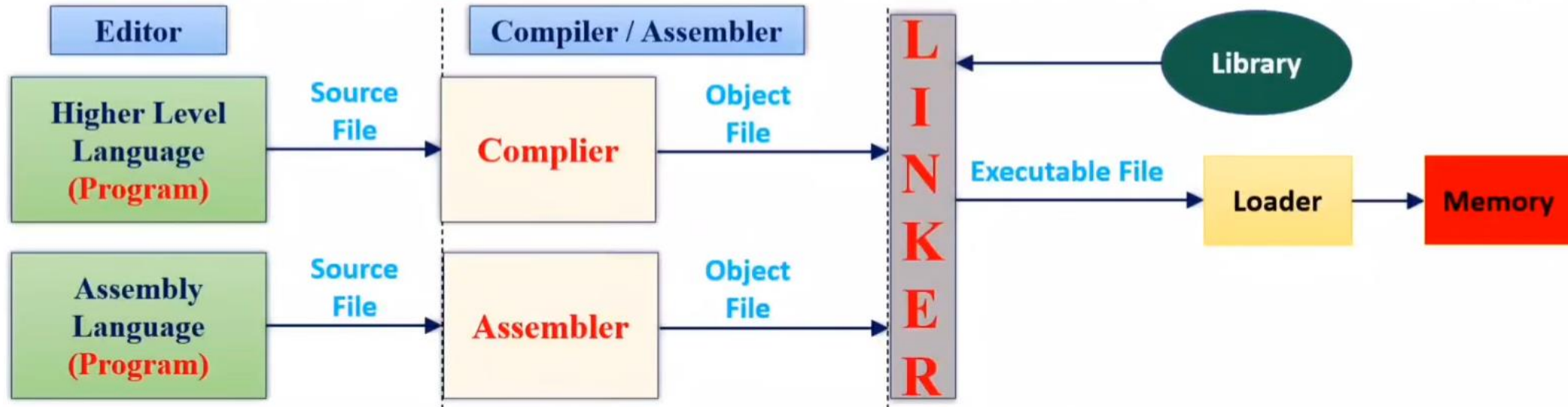
- ☐ In Editor we write programs for Microcontroller.
- ☐ Programs may be written in Assembly Language or Higher Level Language {C Language}.
- ☐ By writing program we generate source file.

System development tool



- ❑ **Assembler** : It is used to convert Assembly language into machine code or object file. It also shows errors if any syntax error is there in program.
- ❑ **Compiler** : It is used to convert Higher Level language into machine code or object file. It also shows errors if any syntax error is there in program. It also gives warnings if it is there with programs.

System development tool



- ☐ **Linker** : It is linking all the object files of compiler and assembler with the use of library.
- ☐ It will generate executable files.
- ☐ **Loader**: It is used to load executable files into the memory of microcontroller.
- ☐ Once program is loaded into memory, microcontroller can execute it as per the requirement of USER.

Data Types

✓ Data Types of 8051 microcontroller

- ❑ 8051 Microcontroller supports Binary, Decimal and Hexadecimal data formats.

Example:

`MOV A,#00110110B` ; A ← 00110110, Binary 00110110 is loaded in A which is equivalent of 36H.

`MOV A,#15` ; A ← 15, Decimal 15 is loaded in A which is equivalent of 0FH.

`MOV A,#15H` ; A ← 15H, Hexadecimal 15H is loaded in A.

Assembler Directives

✓ Assembler Directives of 8051 microcontroller

- ❑ Assembler directives are also referred as pseudo Opcodes. Assembler directives are not instructions, so they are not executed by MPU, it is used to give directions to assembler.

❖ ORG {Origin}

ORG 1000H

- ORG directives is used indicate beginning address.
- After writing ORG 1000H, program or data will be stored at 1000H memory location.
- Some assembler may use .ORG, so you need check that as well.
- If you don't write **H** after number then it will consider decimal address with ORG.

Assembler Directives

❖ END {Terminate}

- It is used to END source file {.asm file}.
- Anything after END directive will be ignored by assembler.

❖ EQU {Equate}

- It is used to define constant without occupying a memory location.
- It is used for constant values in program.

```
pp    EQU 25H           ;constant with pp label
      MOV R1, #pp       ;constant loaded in R1
```

Assembler Directives

❖ DB {Define Byte}

- It is used to define byte data.
- Here, 8 bits {Byte} number can be decimal, Hexadecimal, Binary or ASCII number.

```
ORG 1000H
data1: DB 25           ;data 1 is defined by 25 decimal no.
data2: DB 25H          ;data 2 is defined by 25H Hex no.
data3: DB 00110011B    ;data 3 is defined by 33H binary no.
data4: DB 'A'           ;data 4 is defined by ASCII of A.
data5: DB "EF"          ;data 5 is defined by string of ASCII.
```


Addressing Mode

❖ Addressing mode: The Various formats of specifying the operands are called addressing modes.

✓ Immediate Addressing Mode

❑ In this Addressing mode, data (1byte/2bytes) specified in instruction itself.

❑ Data is specified by '#' symbol before data in the instruction.

Example:

MOV A,#15H ; A ← 15H

MOV DPTR,#1000H ; DPTR ← 1000H

Addressing Mode

✓ Register Addressing Mode

- ☐ In this Addressing mode, data is specified by registers in instruction.
- ☐ The permitted registers are A, R7, R6, ... , R0.

Example:

```
MOV A,R2          ; A ← R2
MOV R2,A          ; R2 ← A
MOV R1,R2          ; Not Allowed with 8051,
```

Addressing Mode

✓ Direct Addressing Mode

- ❑ In this Addressing mode, address of operand is given in instruction.
- ❑ Only Internal RAM and SFR address are allowed.

Example:

MOV A,35H ; A ← [35H]

MOV A,80H ; A ← [80H], Content of port 0 is [80H]

MOV 30H, 35H ; [30H] ← [35H]

Addressing Mode

✓ Indirect Addressing Mode

❑ In this Addressing mode, address of operand will be given by register.

❑ Internal and External RAM can be accessed by this mode.

❖ Internal RAM with 8bits of addressing, "@" is used here

MOV A,@R1 ; $A \leftarrow [R1]$

MOV @R2,A ; $[R2] \leftarrow A$

❖ External RAM with 16bits of addressing by DPTR, "X" is used here

MOVX A,@DPTR ; $A \leftarrow [DPTR]$

MOVX @DPTR,A ; $[DPTR] \leftarrow A$

❖ External RAM with 8bits of addressing by R0 or R1

MOVX A,@R1 ; $A \leftarrow [R1]$, If R1 is 25H then
 $[R1] = [0025H]$

MOVX @R0,A ; $[R0] \leftarrow A$, If R0 is 35H then
 $[R0] = [0035H]$

Addressing Mode

✓ Indexed Addressing Mode

- ❑ This addressing mode is used to access data from code memory {Internal ROM or External ROM}.
- ❑ In instruction, we use 'C' to operate with Indexed Addressing mode.

Example:

MOVC A,@A+DPTR ; $A \leftarrow [A+DPTR]$

MOVC A,@A+PC ; $A \leftarrow [A+PC]$

✓ Addition instructions

- ❑ **ADD** – It will Add Accumulator data with 8 bits & stores result into A.

```
ADD A, #50H      ; A ← A + 50H
ADD A, R1         ; A ← A + R1
ADD A, 17H        ; A ← A + [17H]
ADD A, @R1        ; A ← A + [R1]
```

- ❑ **ADDC** – It will Add Accumulator data with 8 bits along with Carry & stores result into A.

```
ADDC A, #50H      ; A ← A + 50H + Carry
ADDC A, R1         ; A ← A + R1 + Carry
ADDC A, 17H        ; A ← A + [17H] + Carry
ADDC A, @R1        ; A ← A + [R1] + Carry
```


✓ Subtraction instructions

- ❑ SUBB – It will Sub Accumulator data with 8 bits along with Carry & stores result into A.

```
SUBB A, #50H    ; A ← A - 50H - Carry
SUBB A, R1      ; A ← A - R1 - Carry
SUBB A, 17H     ; A ← A - [17H] - Carry
SUBB A, @R1     ; A ← A - [R1] - Carry
```

✓ Increment instructions

- ❑ INC – It will increment Register, Pointer or data of memory locations.

```
INC A    ; A ← A + 1
INC R1   ; R1 ← R1 + 1
INC 25H  ; [25H] ← [25H] + 1
INC @R1  ; [R1] ← [R1] + 1
INC DPTR ; DPTR ← DPTR + 1
```

✓ Decrement instructions

- ❑ DEC – It will decrement Register, Pointer or data of memory locations.

```
DEC A    ; A ← A - 1
DEC R1   ; R1 ← R1 - 1
DEC 25H  ; [25H] ← [25H] - 1
```

✓ Multiplication instruction

- ❑ MUL AB – It will Multiply A and B and Answer will be stored in BA where B hold higher Byte and A hold Lower Byte.

$MUL\ AB ; (B_{Higher\ Byte}\ A_{Lower\ Byte}) \leftarrow A \times B$

✓ Division instruction

- ❑ DIV AB – It will Divide A by B and Answer remainder will be stored in B & Quotient will be stored in A.

$DIV\ AB ; (B_{Remainder}\ A_{Quotient}) \leftarrow A / B$

✓ Decimal Adjustment instruction

- ❑ DA A – It will be used after ADD instruction.
- ❑ It is used to convert that given Hex addition in BCD addition.
- ❑ Normal Addition is done by ADD that is Binary or Hex Addition.
- ❑ After when you use DA A, it adjust that addition in BCD form.
- ❑ DA A performs following adjustments to show given addition in BCD addition.
- ❑ If Lower Nibble > 9 or Auxiliary Carry is 1 then Add 06H with A
- ❑ If Higher Nibble > 9 or Carry is 1 then Add 60H with A

Example:

ADD A, R1 ; $A \leftarrow A + R1$
DA A ; BCD Addition Adjustment

A = 56H
R1 = 23H
A = 79H

A = 36H
R1 = 29H
A = 5FH
06H
A = 65H

AC = 1
A = 38H
R1 = 29H
A = 61H
06H
A = 67H

A = 60H
R1 = 70H
A = D0H
60H
A = 30H
Carry = 1

1 1
A = 99H
R1 = 99H
A = 32H
06H
60H
AL = 98H
Carry = 1

✓ Logic AND instructions

- ❑ ANL – It will perform logic AND in between two 8 bits numbers.

ANL A, #50H	; A ← A AND 50H
ANL A, R1	; A ← A AND R1
ANL A, 17H	; A ← A AND [17H]
ANL A, @R1	; A ← A AND [R1]
ANL 25H, A	; [25H] ← [25H] AND A
ANL 25H, #50H	; [25H] ← [25H] AND 50H

✓ Logic OR instructions

- ❑ ORL – It will perform logic OR in between two 8 bits numbers.

ORL A, #50H	; A ← A OR 50H
ORL A, R1	; A ← A OR R1
ORL A, 17H	; A ← A OR [17H]
ORL A, @R1	; A ← A OR [R1]
ORL 25H, A	; [25H] ← [25H] OR A
ORL 25H, #50H	; [25H] ← [25H] OR 50H

✓ Logic XOR instructions

- ❑ XRL – It will perform logic XOR in between two 8 bits numbers.

XRL A, #50H	; A ← A XOR 50H
XRL A, R1	; A ← A XOR R1
XRL A, 17H	; A ← A XOR [17H]
XRL A, @R1	; A ← A XOR [R1]
XRL 25H, A	; [25H] ← [25H] XOR A
XRL 25H, #50H	; [25H] ← [25H] XOR 50H

✓ Other Logical instructions

- ❑ CPL A – It will perform 1's compliment of A, meant NOT operation of A.
- ❑ CLR A – It will Clear Accumulator. So A = 00H after instruction.

✓ Rotate without Carry instructions

- ☐ **RR A** – It will perform rotate Right of A without carry by 1 bit.
- ☐ If initially A = 46H & Carry = 1



- ☐ After Instruction



- ☐ After Instruction, A = 23H and Carry = 0

- ☐ **RL A** – It will perform rotate Left of A without carry by 1 bit.
- ☐ If initially A = 46H & Carry = 1



- ☐ After Instruction

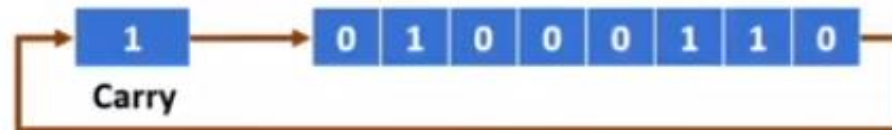


- ☐ After Instruction, A = 8CH and Carry = 0

✓ Rotate with Carry instructions

☐ **RRC A** – It will perform rotate Right of A with carry by 1 bit.

☐ If initially A = 46H & Carry = 1



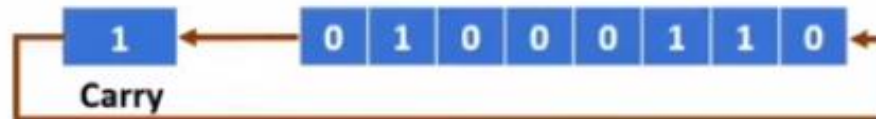
☐ After Instruction



☐ After Instruction, A = A3H and Carry = 0

☐ **RLC A** – It will perform rotate Left of A with carry by 1 bit.

☐ If initially A = 46H & Carry = 1



☐ After Instruction



☐ After Instruction, A = 8DH and Carry = 0

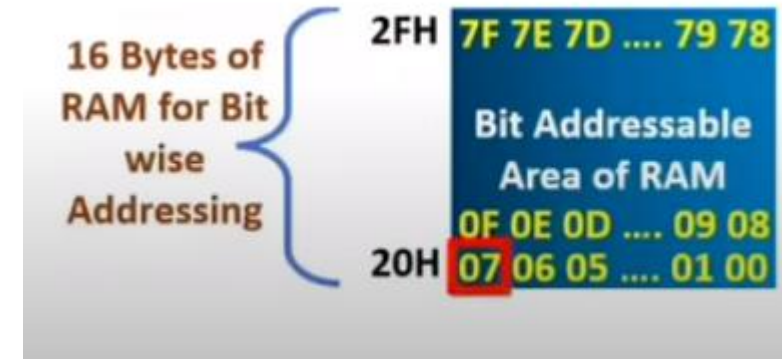
Boolean Instruction

✓ Set, Clear & Complement Carry instructions

- ❑ SETB C – It will make Carry Flag = 1
- ❑ CLR C – It will make Carry Flag = 0
- ❑ CPL C – It will Complement Carry Flag

✓ Set, Clear & Complement Bit instructions

- ❑ SETB – It will make given bit = Logic '1'
 SETB P0.2 ; P0.2 ← 1
 SETB 07H ; 07H bit location in RAM ← 1
- ❑ CLR – It will make given bit = Logic '0'
 CLR P0.2 ; P0.2 ← 0
 CLR 07H ; 07H bit location in RAM ← 0
- ❑ CPL – It will make given bit = complement of initial value
 CPL P0.2 ; P0.2 will get complemented
 CPL 07H ; 07H bit location in RAM will get complemented



Boolean Instruction

✓ Logic AND & OR Bit instructions

- ❑ **ANL C, b** – It will do logic AND between Carry flag and bit b and result will be stored into Carry flag.

ANL C,P0.2 ; C ← C AND P0.2

ANL C,07H ; C ← C AND [07H] bit in RAM

- ❑ **ANL C, /b** – It will do logic AND between Carry flag and complemented bit b and result will be stored into Carry flag.

ANL C,/P0.2 ; C ← C AND {NOT} P0.2

ANL C,/07H ; C ← C AND {NOT} [07H] bit in RAM

- ❑ **ORL C, b** – It will do logic OR between Carry flag and bit b and result will be stored into Carry flag.

ORL C,P0.2 ; C ← C OR P0.2

ORL C,07H ; C ← C OR [07H] bit in RAM

- ❑ **ORL C, /b** – It will do logic OR between Carry flag and complemented bit b and result will be stored into Carry flag.

ORL C,/P0.2 ; C ← C OR {NOT} P0.2

ORL C,/07H ; C ← C OR {NOT} [07H] bit in RAM

Data Transfers

✓ Move instructions

❑ MOV – It will move data from one location to another location.

```
MOV A, #50H      ; A ← 50H
MOV A, R1        ; A ← R1
MOV A, 50H       ; A ← [50H]
MOV A, @R1       ; A ← [R1]
MOV R1, A        ; R1 ← A
MOV R1, #50H     ; R1 ← 50H
MOV R1, 50H      ; R1 ← [50H]
MOV 50H, R1      ; [50H] ← R1
MOV 50H, 40H     ; [50H] ← [40H]
MOV 50H, @R1     ; [50H] ← [R1]
MOV @R1, A       ; [R1] ← A
MOV @R1, #25H    ; [R1] ← 25H
MOV @R1, 25H     ; [R1] ← [25H]
MOV DPTR, #2525H ; DPTR ← 2525H
MOVX A, @R1      ; A ← [00-R1] from External RAM
MOVX A, @DPTR    ; A ← [DPTR] from External RAM
MOVX @R1, A      ; [00-R1] ← A for External RAM
MOVX @DPTR, A    ; [DPTR] ← A for External RAM
MOVC A, @A+DPTR  ; A ← [A+DPTR] for ROM
MOVC A, @A+PC    ; A ← [A+PC] for ROM
```

Stack and Exchange

✓ Stack instructions

- ❑ PUSH – It will store data on stack
- ❑ In PUSH, 1st SP will increment by 1, then it will store data on stack.
- ❑ POP – It will load data from stack
- ❑ In POP, 1st load data from stack, then it will decrement SP by 1.

PUSH R1	; R1 will be stored on stack
PUSH 25H	; [25H] will be stored on stack
POP R1	; R1 will be loaded from stack
POP 25H	; [25H] will be loaded from stack

✓ Exchange instructions

- ❑ XCH – It will exchange the mentioned two data.

XCH A, R0	; A $\longleftrightarrow$ R0
XCH A, 25H	; A $\longleftrightarrow$ [25H]
XCH A, @R1	; A $\longleftrightarrow$ [R1]

- ❑ XCHD A, @Rx – It will exchange lower Nibble of the mentioned two data.

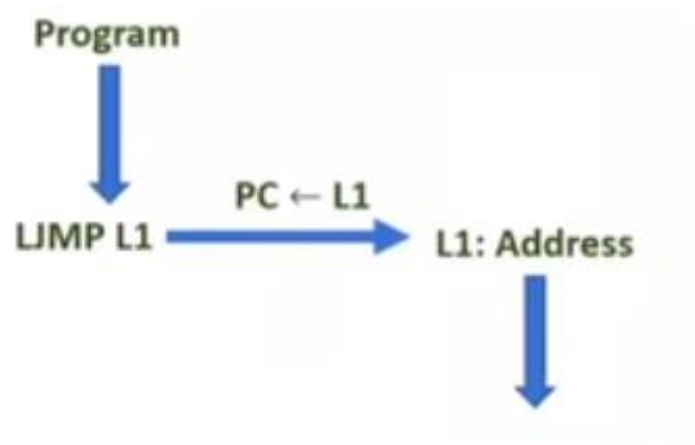
XCHD A, @R1	; A $\longleftrightarrow$ [R1] only Lower Nibble.
-------------	---

Brand Operation

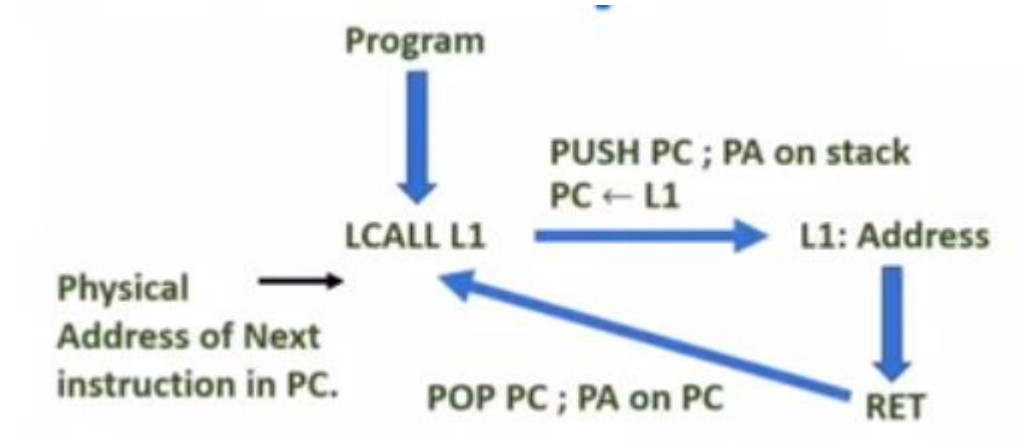
Parameters	SJMP	AJMP	LJMP
Name	▪ SJMP – Short Jump	▪ AJMP – Absolute Jump	▪ LJMP – Long Jump
Syntax	▪ SJMP Label1	▪ AJMP Label2	▪ LJMP Label3
Range	▪ -128 to +127 locations, because of Label1 is 8 bits signed No.	▪ Maximum range is 2KB.	▪ Maximum range is 64KB.
Size	▪ 2 Bytes {1 Opcode + 1 Label}	▪ 2 Bytes {1 Opcode + 1 Label}	▪ 3 Bytes {1 Opcode + 2 Label}
Address calculation	▪ PC = PC of Next instruction + Label1	▪ PC = 1 st 5 bits same of PC + 3bits of AJMP + 8 bits of Label2	▪ PC = 16 bits Label3
Instructions	▪ SJMP ▪ All conditional Jump available	▪ AJMP ▪ ACALL	▪ LJMP ▪ LCALL
Usage	▪ Generally, we use it in same program jump with many conditions and unconditional.	▪ We use it for jump within same page of 2KB. ▪ 64Kb is bisected into 32 pages of 2KB	▪ We use it to jump anywhere in 64KB of 8051 μ C.

JMP and Call in 8051

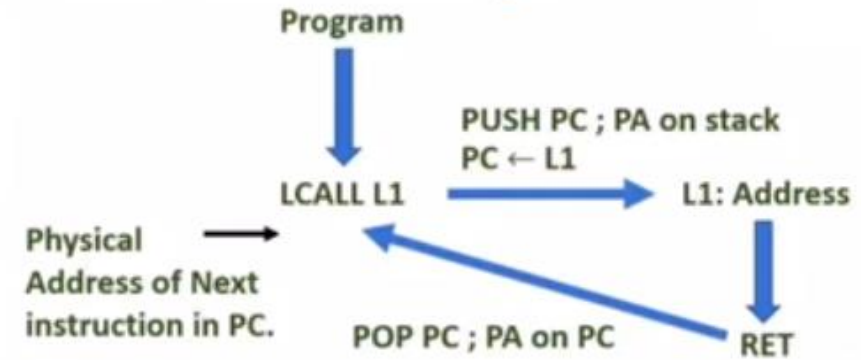
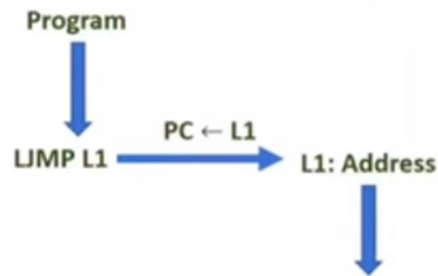
JMP



CALL



JMP and Call



JMP	CALL
❖ Here, we just jump to new location and then we move on with given program.	❖ Here, we jump to new location for subroutine, after subroutine we come back to that location again.
❖ No need of retrieve back to branch location.	❖ By RET at the end of subroutine, it will retrieve back to branch location.
❖ JMP doesn't stores branch location.	❖ CALL stores branch location on Stack.
❖ In 8051, JMP is of three types: ❖ SJMP, AJMP and LJMP	❖ In 8051, CALL is of two types: ❖ ACALL and LCALL
❖ In 8051, SJMP can be conditional and unconditional.	❖ In 8051, we have only unconditional CALL instructions.

RET and RETI

RET	RETI
❖ Used with normal subroutine.	❖ Used with ISR – Interrupt Service Routine
❖ With RET, 8051 just return back to main program from subroutine.	❖ With RETI, 8051 return back to main program + It will Enable Interrupt by making EA = 1.
❖ Operation: POP PC ; PCH ← [SP] ; PCL ← [SP-1] ; SP ← SP - 2	❖ Operation: POP PC ; PCH ← [SP] ; PCL ← [SP-1] ; SP ← SP - 2 EA ← 1

Unconditional Jump instructions

- ☐ **SJMP Label** – It jump to location with respect to Label {8 bits}.
- ☐ Range of SJMP is -128 to +127 locations.
- ☐ Final Address will be $PC = PC + Label$
- ☐ **AJMP Label** – It jump to location with respect to Label {8 bits}.
- ☐ Range of AJMP is 2KB.
- ☐ Final Address will be $PC = 1^{st} \text{ 5 bits of PC} + 3 \text{ bits of AJMP} + Label$.
- ☐ **LJMP Label** – It jump to location at Label {16 bits}.
- ☐ It can jump anywhere in 64KB memory of 8051.
- ☐ Final Address will be $PC = Label$
- ☐ **JMP @A+DPTR** – It will jump to the location $A + DPTR$.

Conditional Jump instructions

- ❑ **DJNZ R3, Label** – It will decrement R3, and jump to the Label only if R3 is not Zero.
- ❑ **DJNZ 25H, Label** – It will decrement [25H], and jump to the Label only if [25H] is not Zero.
- ❑ **CJNE A, #25H, Label** – It will compare A with #25H and jump to the Label only if A and #25H are not equal.
- ❑ **CJNE A, 25H, Label** – It will compare A with [25H] and jump to the Label only if A and [25H] are not equal.
- ❑ **CJNE R2, #25H, Label** – It will compare R2 with #25H and jump to the Label only if R2 and #25H are not equal.
- ❑ **CJNE @R2, #25H, Label** – It will compare [R2] with #25H and jump to the Label only if [R2] and #25H are not equal.
- ❑ **JC Label** – It will jump to Label if with previous instruction, carry flag is 1.
- ❑ **JNC Label** – It will jump to Label if with previous instruction, carry flag is 0.
- ❑ **JZ Label** – It will jump to Label if with previous instruction, Zero flag is 1.
- ❑ **JNZ Label** – It will jump to Label if with previous instruction, Zero flag is 0.

Boolean Conditional Jump Instructions

- ❑ **JB P0.0, Label** – Jump to Label Only if P0.0 = 1
- ❑ **JNB P0.0, Label** – Jump to Label Only if P0.0 = 0
- ❑ **JBC P0.0, Label** – Jump to Label Only if P0.0 = 1 and also make P0.0 = 0